



- 图像预处理的目的:

纠正几何失真



图像坐标变换

提高视觉质量



灰度映射

直方图修正

降低噪声干扰



空域滤波



4.1.1 基本坐标变换

• 1. 变换的表达

$$v' = A v$$

$$v = \begin{bmatrix} x & y & 1 \end{bmatrix}^T \quad A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$v' = \begin{bmatrix} x' & y' & 1 \end{bmatrix}^T$$

图像上各点的新齐次坐标

= 变换矩阵 $\times$ 图像上各点的原齐次坐标



4.1.1 基本坐标变换

• 3. 尺度变换

$$S = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad S^{-1} = \begin{bmatrix} 1/S_x & 0 & 0 \\ 0 & 1/S_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

原图



尺度变换



4.1.1 基本坐标变换

• 2. 平移变换

$$T = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix} \quad T^{-1} = \begin{bmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

原图



平移变换



4.1.1 基本坐标变换

• 4. 旋转变换

$$R = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad R^{-1} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

原图



旋转变换





4.1.2 几何失真校正

1. 空间变换

$$x' = s(x, y)$$

$$y' = t(x, y)$$

$$s(x, y) = k_1x + k_2y + k_3$$

$$t(x, y) = k_4x + k_5y + k_6$$

线性失真

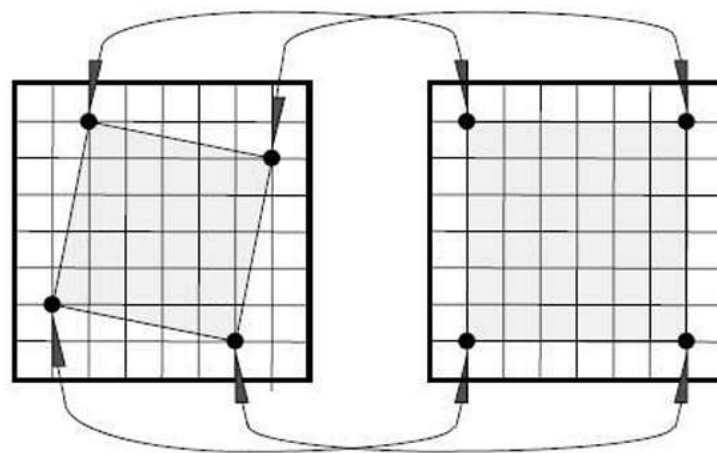


图 4.1.1 失真图和校正图的对应点

$$s(x, y) = k_1x + k_2y + k_3xy + k_4$$

$$t(x, y) = k_5x + k_6y + k_7xy + k_8$$

双线性失真

```
%% 自定义旋转函数
function rotated = rotate_image(img, angle)
[rows, cols] = size(img);
% 计算旋转角度（弧度）
theta = angle * pi / 180;...
```

显示思路

这是一个非常经典的图像处理问题。在计算机视觉和图像几何变换中，映射方式主要分为两种：**反向映射（Backward Mapping）** 和 **正向映射（Forward Mapping）**。

你给出的代码使用的是**反向映射**，这也是 OpenCV、PyTorch 等主流库和框架中处理仿射变换（如旋转、缩放）的标准做法。

下面为你详细梳理正向映射（正向插值）的原理、实现方式，以及为什么我们通常不采用它。

1. 核心逻辑的区别

- 反向映射（你的代码）**：遍历新图像的每一个像素位置，通过逆变换矩阵计算出它在原图像中的浮点坐标，然后在原图像上进行双线性插值，“借”一个颜色值过来。
- 正向映射（你要问的）**：遍历原图像的每一个像素位置，通过正向变换矩阵计算出它在新图像中的浮点坐标，然后把它的颜色值“送”过去。

2. 正向映射的数学表达

假设原图像中的像素坐标为 (x, y) ，绕中心逆时针旋转 θ 角度后的新浮点坐标为 (u, v) 。正向变换公式为：

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

这里会产生一个核心问题：算出来的 (u, v) 通常是带有小数的浮点数，而图像的像素坐标必须是整数。

3. 最朴素的正向映射（最近邻）及其缺陷

如果直接对 (u, v) 进行四舍五入取整： $u' = \text{round}(u)$, $v' = \text{round}(v)$ ，然后赋值 `new_img(u', v') = img(x, y)`，会导致两个严重的视觉缺陷：

- 空洞现象（Holes）**：由于旋转后的图像通常比原图像大，加上取整误差，新图像中会有一些像素永远接收不到原图像映射过来的值。这些像素会保持初始化的颜色（通常是黑色），导致画面上出现规则黑色网格或斑点。
- 像素重叠（Overlaps）**：原图像中相邻的两个像素，经过旋转和取整后，可能会被映射到新图像的同一个坐标上，导致后计算的像素覆盖掉先计算的像素，丢失信息。

4. 正向插值怎么做？（Splatting / 像素能量分布）

为了解决上述问题，正向插值不能简单地“四舍五入”，而是要采用一种叫做 **Splatting**（溅射法 / 抛雪球法）的策略。

既然原像素 (x, y) 被映射到了浮点坐标 (u, v) ，我们就把原像素的颜色值（或者说能量）按距离比例分配给新图像中 (u, v) 周围最近的 4 个整数像素点。

具体步骤如下：

1. 确定周围的 4 个目标像素：
 设浮点坐标为 (u, v) ，它周围的四个整数坐标分别是：
 $u_0 = \lfloor u \rfloor, \quad u_1 = u_0 + 1$
 $v_0 = \lfloor v \rfloor, \quad v_1 = v_0 + 1$
2. 计算分配权重（类似双线性插值的权重）：
 计算小数部分： $dx = u - u_0, \quad dy = v - v_0$
 四个点分到的权重分别为：
 $w_{00} = (1 - dx)(1 - dy)$ （左上）
 $w_{10} = dx(1 - dy)$ （右上）
 $w_{01} = (1 - dx)dy$ （左下）
 $w_{11} = dx \cdot dy$ （右下）

3. 累加像素值与权重：

因为可能会有多个原像素把能量分配给同一个新像素（重叠问题），我们必须在目标图像中维护两个矩阵：一个是颜色累加矩阵 I_{new} ，一个是权重累加矩阵 W_{new} 。

对于原图像中的像素值 $P = I(x, y)$ ，进行如下累加：
 $I_{new}(u_0, v_0) += P \cdot w_{00}, \quad W_{new}(u_0, v_0) += w_{00}$
 $I_{new}(u_1, v_0) += P \cdot w_{10}, \quad W_{new}(u_1, v_0) += w_{10}$
 $I_{new}(u_0, v_1) += P \cdot w_{01}, \quad W_{new}(u_0, v_1) += w_{01}$
 $I_{new}(u_1, v_1) += P \cdot w_{11}, \quad W_{new}(u_1, v_1) += w_{11}$

4. 最终归一化：

遍历完原图像所有像素后，目标图像的最终颜色需要用颜色累加值除以总权重（避免重叠区域过亮）：

$$\text{Result}(i, j) = \frac{I_{new}(i, j)}{W_{new}(i, j)}$$

（注：如果某个像素的 $W_{new} = 0$ ，说明完全没有原像素映射到这里，依然会是一个空洞，通常需要用周围像素的均值再做一次后处理填补。）

总结：为什么工程上偏爱反向映射？

通过对比可以发现，正向插值的逻辑极其繁琐：

- 不仅需要维护额外的权重矩阵（消耗内存）。
- 需要大量的加法操作和最后的除法归一化（计算复杂度高）。
- 即使做了能量分配，边缘处仍可能产生难以处理的微小空洞。

而你代码中使用的反向映射，因为是从目标图像倒推回原图像，天然保证了目标图像的每一个像素都被计算了一次且仅计算一次，彻底杜绝了空洞和重叠问题，而且无需额外的累加和归一化步骤。这就是为什么无论是在传统的图像算法中，还是在深度学习的张量空间变换（如 Spatial Transformer Networks）中，反向映射结合双线性插值都是绝对的主流。



4.2.2 灰度映射示例

灰度映射技术的关键是根据增强要求设计映射函数

扩展低灰度范围
压缩高灰度范围

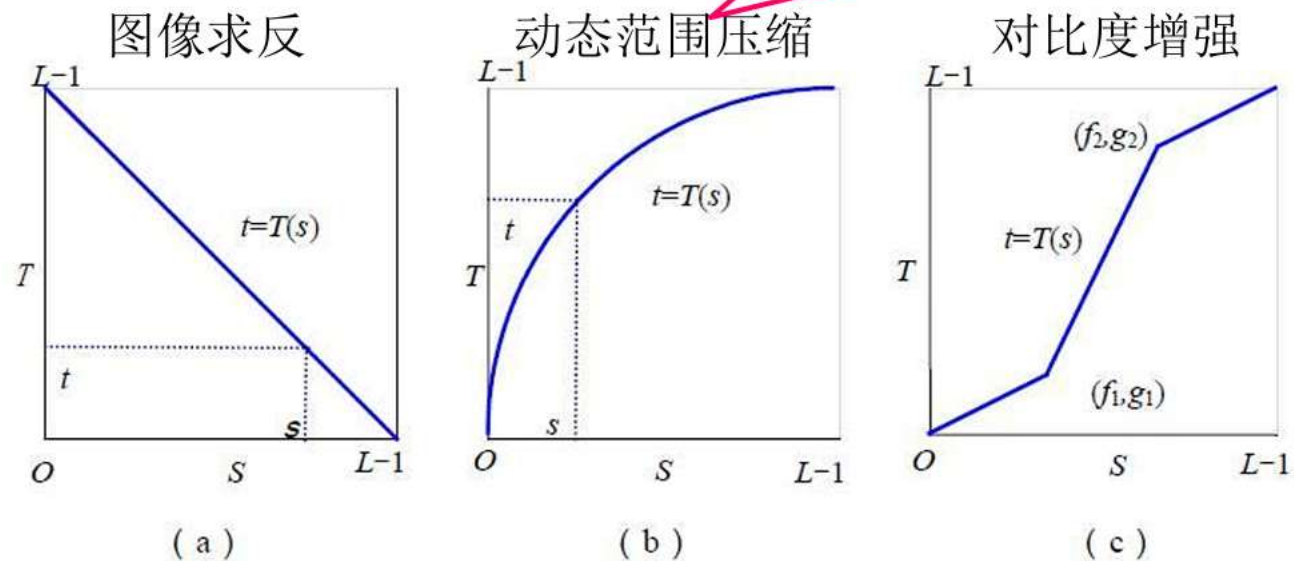


图 4.2.2 若干典型的灰度映射函数的示例

4.2.2 灰度映射示例

思考：如何判断是压缩还是扩展

对输入图像灰度作线性扩张或压缩，映射函数为一个直线方程，其表达式如下：

$$t = a \times s + b;$$

其中：

a相当于变换直线的斜率，b相当于截距；

$$b = 0: \begin{cases} a > 1 & \text{----对比度扩张} \\ a < 1 & \text{----对比度压缩} \\ a = 1 & \text{----相当于复制} \end{cases}$$

b $\neq$ 0: 灰度偏置

在冈萨雷斯的数字图像处理这本书里关于对比度的定义是什么呢？

◆ 显示思路 ▾

⋮

在拉斐尔·C·冈萨雷斯（Rafael C. Gonzalez）和理查德·E·伍兹（Richard E. Woods）所著的经典教材《数字图像处理》（Digital Image Processing）中，关于对比度（Contrast）并没有局限于一个单一的数学公式，而是从物理属性、统计特征以及人类视觉感知等多个维度进行了递进式的定义和阐述。

书中关于对比度的核心定义主要体现在以下几个层面：

1. 基础物理与强度定义（最直接的定义）

在书中讨论图像的基本属性时，给出了对比度最直观的定义：

对比度是指一幅图像中最高和最低灰度级（或强度级别）之间的差异。

- **高对比度（High Contrast）：** 图像中最高和最低灰度级之间的差值很大（即包含了极暗和极亮的区域），且灰度级跨度广。
- **低对比度（Low Contrast）：** 图像的灰度值集中在一个非常狭窄的范围内，最高与最低强度级别之间的差异很小。这种图像视觉上通常显得灰蒙蒙的，缺乏层次感，或者整体过亮/过暗。



作业

- 练习题:

- 4.1

4.1 设给定平移量 $(2, 5)$ ，并用 2 和 5 作为放缩因子沿 X 和 Y 轴进行尺度变换，分别计算对图像点 $(2, 5)$ 先平移变换后尺度变换和先尺度变换后平移变换所得到的结果，并进行比较和讨论。

- 4.2

4.2 给出实现对一个像素先平移，再旋转，最后尺度变换的变换矩阵。